

EDS Lab Assignments

Practical 3

Name: Dhanashri Mamidwar

PRN: 202501040059

3.1.1 Numpy Array Operations

Problem Statement:

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Code:

```
import numpy as np
rows, cols = map(int, input().split())
elements = []

for _ in range(rows):
    elements.extend(map(int, input().split()))

array = np.array(elements).reshape(rows, cols)
print(array)
print(array.ndim)
print(array.shape)
print(array.size)
```

Output:

The screenshot displays the CODETANTRA IDE interface. On the left, a sidebar shows the file explorer with a file named '3.1.1. Numpy Array Operations'. The main editor area contains a Python script that reads user input for the number of rows and columns, then reads the array elements and reshapes them into a 2D NumPy array. The script prints the array and its dimensions. Below the code editor, a terminal window shows the output of the program. On the right, a test runner panel indicates that 2 out of 2 shown test cases passed, with a total of 10 out of 10 hidden test cases passed. The test cases show the expected output (a 2D array) and the actual output (the same 2D array).

3.2.1 Numpy: Matrix Operations

Problem Statement:

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** (`matrix_a + matrix_b`)
2. **Subtraction** (`matrix_a - matrix_b`)
3. **Element-wise Multiplication** (`matrix_a * matrix_b`)
4. **Matrix Multiplication** (`matrix_a · matrix_b`)
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.

4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code:

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# Addition
addition_result = matrix_a + matrix_b
print("Addition (A + B):")
print(addition_result)

# Subtraction
subtraction_result = matrix_a - matrix_b
print("Subtraction (A - B):")
print(subtraction_result)

# Multiplication (element-wise)
elementwise_multiplication_result = matrix_a * matrix_b
print("Element-wise Multiplication (A * B):")
print(elementwise_multiplication_result)

# Matrix multiplication (dot product)
matrix_multiplication_result = np.dot(matrix_a, matrix_b)
print("A dot B:")
print(matrix_multiplication_result)

# Transpose
transpose_a = matrix_a.T
print("Transpose of A:")
print(transpose_a)
```

Output:

The first screenshot shows the initial code implementation in the CODETANTRA IDE. The task is to perform five matrix operations on two 3x3 matrices, `matrix_a` and `matrix_b`, which are input by the user and converted to NumPy arrays. The operations are: 1. Addition (`matrix_a + matrix_b`), 2. Subtraction (`matrix_a - matrix_b`), 3. Element-wise Multiplication (`matrix_a * matrix_b`), 4. Matrix Multiplication (`matrix_a * matrix_b`), and 5. Transpose of Matrix A (`matrix_a.T`). The code uses NumPy's `np.array` for conversion and various operators for the operations. The output format specifies the order of results: Addition, Subtraction, Element-wise Multiplication, Matrix Multiplication, and Transpose of Matrix A.

The second screenshot shows the same code after running the tests. The test results indicate that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test case details show the expected and actual outputs for each operation, confirming the correctness of the implementation. For example, for the addition test case, the expected output is `[[ 2 3 4], [ 5 6 7], [ 8 9 10]]` and the actual output matches this.

3.2.2 Numpy: Horizontal and Vertical Stacking of Arrays

Problem Statement:

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code:

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Perform vertical stacking (vstack)
b = np.vstack((arr1, arr2))
print("Vertical Stack:")
print(b)
```

Output:

3.2.2. NumPy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking** Stack the two matrices horizontally (side by side).
- Vertical Stacking** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```

1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 hstack_result = np.hstack((arr1, arr2))
12 print("Horizontal Stack:")
13 print(hstack_result)
14
15 # Perform vertical stacking (vstack)
16 vstack_result = np.vstack((arr1, arr2))
17 print("Vertical Stack:")
18 print(vstack_result)

```

Average time: 0.023 s, Maximum time: 0.025 s, 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

Test case 1

Expected output	Actual output
Enter Array1:	Enter Array1:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Array2:	Enter Array2:
4 5 6	4 5 6
7 8 9	7 8 9

3.2.3 NumPy: Custom Sequence Generation

Problem Statement:

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

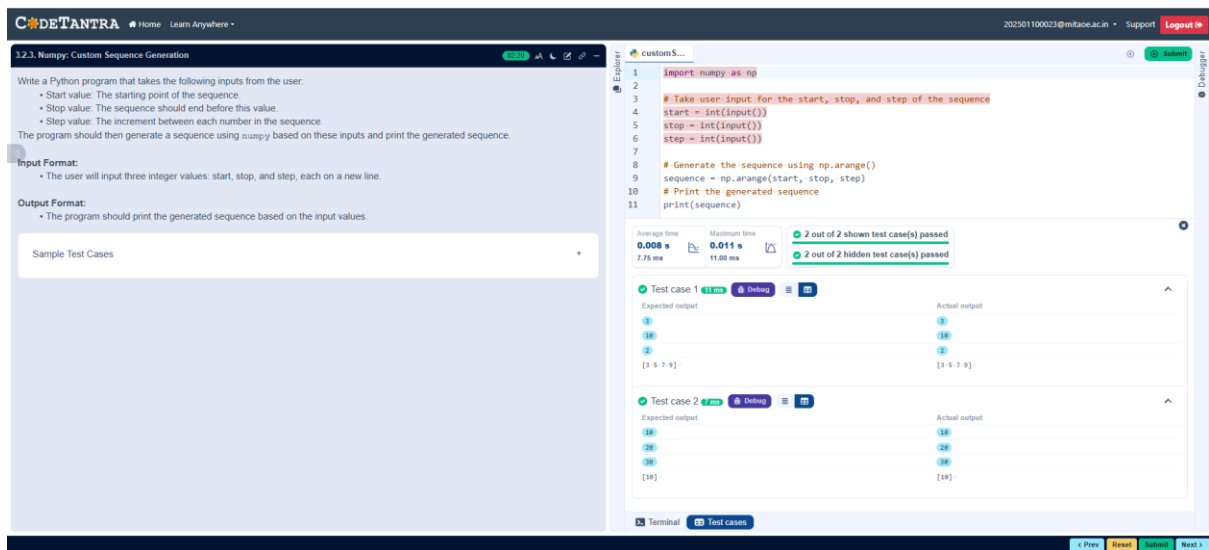
Code:

```

import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using np.arange()
a = np.arange(start, stop, step)
# Print the generated sequence
print(a)

```

Output:



3.2.4 Numpy: Arithmetic and Statistical Operations, Mathematical Operations and Bitwise Operators

Problem Statement:

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code:

```

import numpy as np

def array_operations(A, B):
    # Convert A and B to NumPy arrays
    A = np.array(A)
    B = np.array(B)

    # Arithmetic Operations
    sum_result = A + B
    diff_result = A - B
    prod_result = A * B

    # Statistical Operations
    mean_A = np.mean(A)
    median_A = np.median(A)
    std_dev_A = np.std(A)

    # Bitwise Operations
    and_result = A & B
    or_result = A | B
    xor_result = A ^ B

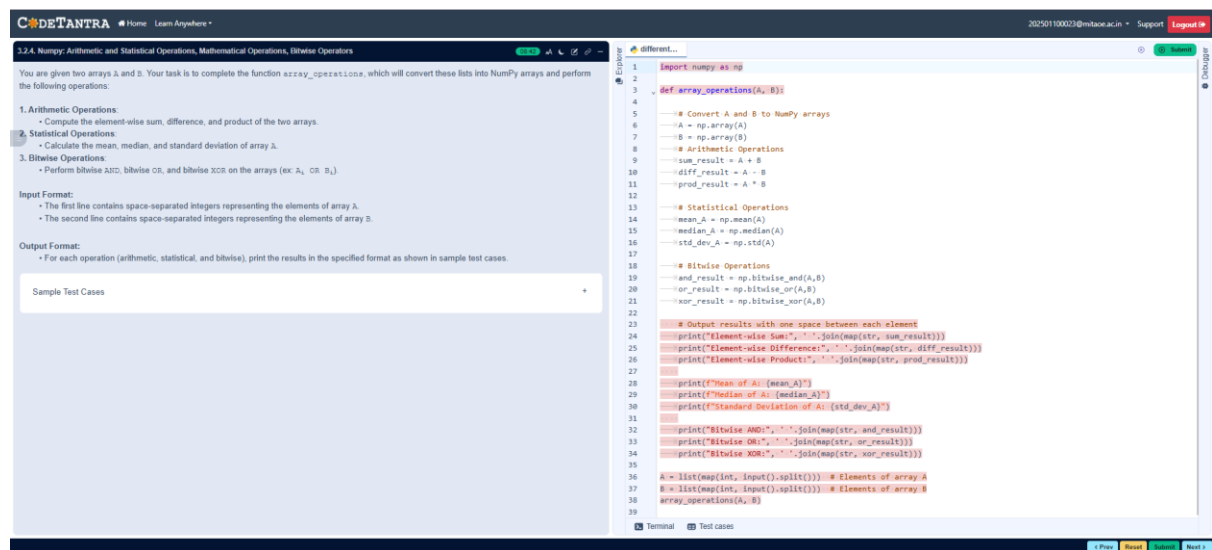
    # Output results with one space between each element
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str, prod_result)))

    print(f"Mean of A: {mean_A}")
    print(f"Median of A: {median_A}")
    print(f"Standard Deviation of A: {std_dev_A}")
    print("Bitwise AND:", ' '.join(map(str, and_result)))
    print("Bitwise OR:", ' '.join(map(str, or_result)))
    print("Bitwise XOR:", ' '.join(map(str, xor_result)))

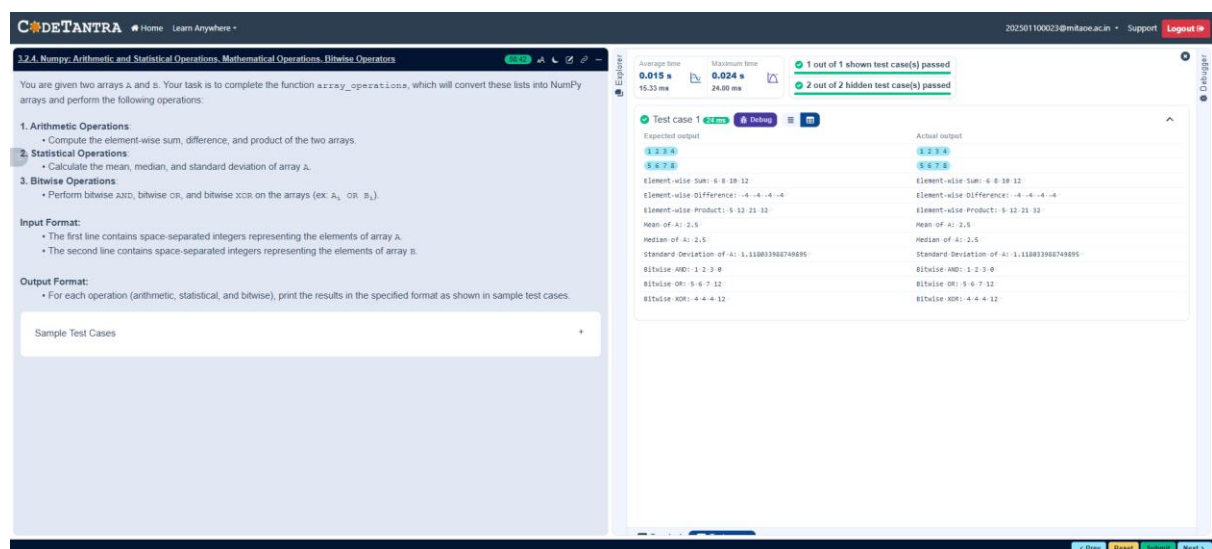
A = list(map(int, input().split())) # Elements of array A
B = list(map(int, input().split())) # Elements of array B
array_operations(A, B)

```

Output:



```
1 import numpy as np
2
3 def array_operations(A, B):
4
5     # Convert A and B to NumPy arrays
6     A = np.array(A)
7     B = np.array(B)
8
9     # Arithmetic Operations
10    sum_result = A + B
11    diff_result = A - B
12    prod_result = A * B
13
14    # Statistical Operations
15    mean_A = np.mean(A)
16    median_A = np.median(A)
17    std_dev_A = np.std(A)
18
19    # Bitwise Operations
20    and_result = np.bitwise_and(A, B)
21    or_result = np.bitwise_or(A, B)
22    xor_result = np.bitwise_xor(A, B)
23
24    # Output results with one space between each element
25    print("Element-wise Sum:", " ".join(map(str, sum_result)))
26    print("Element-wise Difference:", " ".join(map(str, diff_result)))
27    print("Element-wise Product:", " ".join(map(str, prod_result)))
28
29    print("Mean of A: {}".format(mean_A))
30    print("Median of A: {}".format(median_A))
31    print("Standard Deviation of A: {}".format(std_dev_A))
32
33    print("Bitwise AND:", " ".join(map(str, and_result)))
34    print("Bitwise OR:", " ".join(map(str, or_result)))
35    print("Bitwise XOR:", " ".join(map(str, xor_result)))
36
37    A = list(map(int, input().split())) # Elements of array A
38    B = list(map(int, input().split())) # Elements of array B
39    array_operations(A, B)
```



```
Test case 1
Expected output:
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 6 12 21 32
Mean of A: 2.5
Median of A: 2.5
Standard Deviation of A: 1.118033988749895
Bitwise AND: 1 2 3 0
Bitwise OR: 5 6 7 12
Bitwise XOR: 4 4 4 12
Actual output:
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 6 12 21 32
Mean of A: 2.5
Median of A: 2.5
Standard Deviation of A: 1.118033988749895
Bitwise AND: 1 2 3 0
Bitwise OR: 5 6 7 12
Bitwise XOR: 4 4 4 12
```

3.2.5 Numpy: Copying and Viewing Arrays

Problem Statement:

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
- After modifying the copy:
Original array after modifying copy: <original_array>
Copy array: <copy_array>

Code:

```
import numpy as np
inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a sidebar shows the file explorer with a file named 'copyAndView.py'. The main editor area contains the following Python code:

```
1 import numpy as np
2 inputlist = list(map(int,input().split(" ")))
3
4 # Original array
5 original_array = np.array(inputlist)
6
7 # Create a view
8 view_array = original_array.view()
9
10 # Create a copy
11 copy_array = original_array.copy()
12
13 # Modify the view
14 view_array[0] = 99
15
16 print("Original array after modifying view:", original_array)
17 print("View array:", view_array)
18
19 # Modify the copy
20 copy_array[1] = 88
21
22 print("Original array after modifying copy:", original_array)
23 print("Copy array:", copy_array)
```

Below the code editor, the output of the program is shown. It includes the execution time (0.005 s), the output of the program, and the results of two test cases. The output shows the original array after modifying the view, the view array, the original array after modifying the copy, and the copy array.

Test case 1: 2 out of 2 shown test case(s) passed. 2 out of 2 hidden test case(s) passed.

Test case 2: 2 out of 2 shown test case(s) passed. 2 out of 2 hidden test case(s) passed.

3.2.6 Searching, Sorting, Counting, Broadcasting

Problem Statement:

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching:** Find the indices where search_value appears in array1 and print these indices.
- 2. Counting:** Count how many times count_value appears in array1 and print the count.
- 3. Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- 4. Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Code:

```
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))
```

```
# Find indices where value matches in array1
a = np.where(array1 == search_value)[0]
print(a)
```

```
# Count occurrences in array1
b = np.count_nonzero(array1 == count_value)
print(b)
```

```
# Broadcasting addition
c = array1 + broadcast_value
print(c)
```

```
# Sort the first array
d = np.sort(array1)
print(d)
```

Output:

The screenshot displays a coding environment with a problem description on the left and a code editor on the right. The problem description for '12.6. Numpy: Searching, Sorting, Counting, Broadcasting' includes instructions on how to use the code, input/output formats, and sample test cases. The code editor shows a Python script that implements these operations using NumPy. The test results section at the bottom indicates that 2 out of 2 shown test cases passed, with a comparison between expected and actual outputs for each case.

Problem Description:

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching:** Find the indices where `search_value` appears in `array1` and print these indices.
- Counting:** Count how many times `count_value` appears in `array1` and print the count.
- Broadcasting:** Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
- Sorting:** Sort `array1` in ascending order and print the sorted array.

Input Format:

- A single line containing space-separated integers representing `array1`.
- An integer `search_value` represents the value to search for in the array.
- An integer `count_value` represents the value to count in the array.
- An integer `broadcast_value` represents the value to add to each element of the array.

Output Format:

- The indices where `search_value` occurs in `array1`.
- The count of occurrences of `count_value` in `array1`.
- The array after adding the `broadcast_value` to each element.
- The sorted array.

Sample Test Cases

Code Editor:

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 indices = np.where(array1 == search_value)[0]
13 print(indices)
14
15 # Count occurrences in array1
16 count = np.count_nonzero(array1 == count_value)
17 print(count)
18
19 # Broadcasting addition
20 broadcasted_array = array1 + broadcast_value
21 print(broadcasted_array)
22
23 # Sort the first array
24 sorted_array = np.sort(array1)
25 print(sorted_array)
```

Test Results:

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1:

Expected output	Actual output
0 1 2 3 4	0 1 2 3 4
Value to search: 2	Value to search: 2
Value to count: 2	Value to count: 2
Value to add: 2	Value to add: 2
[0 1 2]	[0 1 2]
3	3
[2 3 3 4 4 4]	[2 3 3 4 4 4]
[1 1 1 2 2 2]	[1 1 1 2 2 2]

3.2.7 Student Data Analysis and Operations

Problem Statement:

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.

- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Code:

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

roll = a[:, 0].astype(float)
s1 = a[:, 1]
s2 = a[:, 2]
s3 = a[:, 3]

# 1. Print all student details
print("All student Details:\n", a)

# 2. print total students
print("Total Students:", len(a))

# 3. Print all student Roll numbers
print("All Student Roll Nos", roll)

# 4. Print subject 1 marks
print("Subject 1 Marks", s1)

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(s2))

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(s3))

# 7. Print All subject marks
print("All subject marks:", a[:, 1:])

# 8. print Total marks of students
total = s1 + s2 + s3
print("Total Marks", total)

# 9. print average marks of each student
```

```

print(np.round(total/3, 1))

# 10. print average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis = 0))

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2", np.mean([s1, s2], axis = 1))

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3", np.mean([s1, s3], axis = 1))

# 13. print Roll number who got maximum marks in Subject 3
print("Roll no who got maximum marks in Subject 3", float(roll[np.argmax(s3)]))

# 14. print Roll number who got minimum marks in Subject 2
print("Roll no who got minimum marks in Subject 2", float(roll[np.argmin(s2)]))

# 15. print Roll number who got 24 marks in Subject 2
print("Roll no who got 24 marks in Subject 2", roll[s2 == 24].astype(float).reshape(1, -1))

# 16. print count of students who got marks in Subject 1 < 40
print("Count of students who got marks in Subject 1 < 40", np.sum(s1 < 40))

# 17. print count of students who got marks in Subject 2 > 90
print("Count of students who got marks in Subject 2 > 90:", np.sum(s2 > 90))

# 18. print count of students in each subject who got marks >= 90
print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90,
axis = 0))

# 19. print count of subjects in which each student got marks >= 90
print("Roll no:", roll)
print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis =
1))

# 20. Print S1 marks in ascending order
print(np.sort(s1))

# 21. Print S1 marks >= 50 and <= 90
print(a[(s1 >= 50) & (s1 <= 90)].astype(float))
print(a.astype(float))

# 22. Print the index position of marks 79
print(np.where(s1 == 79))

```

CDEBTANTRA
Home
Learn Python Here

202501100023@mlstacks.ai
In
Logout

3.2.7. Student Data Analysis and Operations

```

1  # Import numpy as np
2
3  # np.loadtxt('sample.csv', delimiter=',', skiprows=1)
4
5  # 1. Print all student details
6  print('All student details:\n', a )
7
8  # 2. print total students
9  print('Total Students',a.shape[0] )
10
11 # 3. Print all student Roll numbers
12 print('All Student Roll Nos',a[:,0] )
13
14 # 4. Print subject 1 marks
15 print('Subject 1 Marks',a[:,1] )
16
17 # 5. print minimum marks of Subject 2
18 print('Min marks in Subject 2',np.min(a[:,2]))
19
20 # 6. print maximum marks of Subject 3
21 print('Max marks in Subject 3',np.max(a[:,3] ) )
22
23 # 7. Print All subject marks
24 print('All subject marks', a[:,1:] )
25
26 # 8. Print Total marks of students
27 print('Total Marks', np.sum(a[:,1:],axis=0) )
28
29 # 9. print average marks of each student
30 np.round(np.mean(a[:,1:],axis=1),3)
31
32 # 10. print average marks of each subject
33 print('Average Marks of each subject', np.round(np.mean(a[:,1:],axis=0),1))
34
35 # 11. print average marks of S1 and S2
36 print('Average Marks of S1 and S2', np.round(np.mean(a[:,1,2:],axis=0),1))
37
38 # 12. print average marks of S1 and S3
39 print('Average Marks of S1 and S3', np.round(np.mean(a[:, 1,3:],axis=0),1))

```

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >= 90 in each subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >= 90: Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39

```

Terminal

Test cases

CyberTANTRA | Home | Learn More >
202501100023@mlhack.ac.in - Support Logout >

3.2. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >= 90 in each subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >= 90: Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

```

Operations...
37 # 12. Print average marks of S1 and S3
38 print("Average Marks of S1 and S3", np.round(np.mean(a[:, [1,3]],axis=0),1))
39
40 # 13. Print Roll number who got maximum marks in Subject 3
41
42 print("Roll no who got maximum marks in Subject 3", np.argmax(a[:,3],0))
43
44 # 14. Print Roll number who got minimum marks in Subject 2
45
46 print("Roll no who got minimum marks in Subject 2", np.argmin(a[:,2],0))
47
48 # 15. Print Roll number who got 24 marks in Subject 2
49
50 print("Roll no who got 24 marks in Subject 2", a[a[:,2] == 24,0].reshape(-1,1))
51
52 # 16. Print count of students who got marks in Subject 1 < 40
53 print("Count of students who got marks in Subject 1 < 40")
54
55 # 17. Print count of students who got marks in Subject 2 > 90
56 print("Count of students who got marks in Subject 2 > 90", np.sum(a[:,2] > 90))
57
58 # 18. Print count of students in each subject who got marks >= 90
59 print("Count of students in each subject who got marks >= 90", np.sum(a[:,1:] >= 90, axis=0))
60
61 # 19. Print count of subjects in which each student got marks >= 90
62 print("Roll no.", a[:,0])
63 print("Count of subjects in which student got marks >= 90:", np.sum(a[:,1:] >= 90, axis=1))
64
65 # 20. Print S1 marks in ascending order
66 print(np.sort(a[:, 3]))
67
68 # 21. Print S1 marks >= 50 and <= 90
69 print(a[a[:,1] >= 50] & (a[:,1] <= 90))
70 print(a)
71
72 # 22. Print the Index position of marks 79
73 print(np.where(a[:,1] == 79))
74
75
Terminal Test cases

```

CODETANTRA
Home Learn Ask Help

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 3: Compute the average marks for Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored >=90 in every subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored >=90: Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Sample Test Cases

Average time
0.023 s
23.00 ms
Maximum time
0.023 s
23.00 ms
1 out of 1 shown test case(s) passed

Test case 1
Status
Python

Actual output	Expected output
All student details: [[101, 67, 77, 88], [102, 78, 88, 77], [103, 45, 56, 89], [104, 88, 98, 45], [105, 78, 88, 99], [106, 68, 78, 36], [107, 79, 52, 45], [108, 45, 45, 77], [109, 89, 52, 88], [110, 79, 24, 33]]	All student details: [[101, 67, 77, 88], [102, 78, 88, 77], [103, 45, 56, 89], [104, 88, 98, 45], [105, 78, 88, 99], [106, 68, 78, 36], [107, 79, 52, 45], [108, 45, 45, 77], [109, 89, 52, 88], [110, 79, 24, 33]]
Total Students: 10	Total Students: 10
All Student Roll Nos [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]	All Student Roll Nos [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
Subject 1 Marks [67, 78, 45, 88, 78, 68, 79, 46, 89, 79]	Subject 1 Marks [67, 78, 45, 88, 78, 68, 79, 46, 89, 79]
Min marks in subject 2 22.0	Min marks in subject 2 22.0
Max marks in subject 3 99.0	Max marks in subject 3 99.0
All subject marks [[67, 77, 88], [78, 88, 77], [45, 56, 89], [88, 98, 45], [78, 88, 99], [68, 78, 36], [79, 52, 45], [45, 45, 77], [89, 52, 88], [79, 24, 33]]	All subject marks [[67, 77, 88], [78, 88, 77], [45, 56, 89], [88, 98, 45], [78, 88, 99], [68, 78, 36], [79, 52, 45], [45, 45, 77], [89, 52, 88], [79, 24, 33]]
Total Marks [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]	Total Marks [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
[77.5, 82, 63.7, 77, 88.5, 67.45, 63.56, 69.5, 45.5]	[77.5, 82, 63.7, 77, 88.5, 67.45, 63.56, 69.5, 45.5]
Average Marks of each subject [71.7, 59.8, 67.7]	Average Marks of each subject [71.7, 59.8, 67.7]
Average Marks of S1 and S2 [71.7, 59.8]	Average Marks of S1 and S2 [71.7, 59.8]